

Understanding Customer Churn Prediction: A Business-Focused Tutorial using Google Colab

Su Nguyen

December 2, 2025

Contents

1	Introduction: Why Predict Customer Churn?	2
2	Setting Up: Loading Libraries and Data	2
2.1	Loading the Data	2
3	Getting to Know Your Data (Exploratory Data Analysis - Part 1)	3
3.1	Basic Checks	3
3.2	Interpreting the Findings	3
4	Visualizing the Data (Exploratory Data Analysis - Part 2)	4
4.1	Histogram: Distribution of Tenure	4
4.2	Box Plot: Monthly Charges vs. Churn	5
5	Preparing Data for Modeling (Preprocessing)	6
5.1	Handling Data Issues	6
5.2	Encoding Categorical Data: One-Hot Encoding	6
6	Understanding Relationships (Correlation)	7
7	Simplifying Data (Dimensionality Reduction with PCA)	8
8	Building Predictive Models	9
8.1	Train-Test Split: The Golden Rule	9
8.2	Model 1: Logistic Regression	9
8.3	Model 2: Random Forest	10
8.4	Model 3: Decision Tree	10
8.5	Interpreting the Decision Tree: Finding Business Rules	11
9	Using Colab's Code Generation	11
10	Conclusion: From Data to Business Action	11

1 Introduction: Why Predict Customer Churn?

Customer churn, or customer attrition, is when customers stop doing business with a company. For any subscription-based business (like telecom, streaming services, SaaS), retaining existing customers is often more cost-effective than acquiring new ones. Understanding *why* customers leave and *who* is likely to leave allows businesses to take proactive steps to retain them.

This tutorial walks through a typical data science project focused on predicting customer churn using a real-world dataset from IBM Telco. We'll cover:

1. **Exploratory Data Analysis (EDA):** Getting to know the data, finding patterns, and understanding the characteristics of customers who churn versus those who don't.
2. **Data Preparation:** Cleaning and transforming the data so it's suitable for analysis and modeling.
3. **Dimensionality Reduction (Optional but useful):** Simplifying the data using techniques like Principal Component Analysis (PCA) for visualization or potentially faster modeling.
4. **Predictive Modeling:** Building models (like Logistic Regression, Decision Trees, Random Forests) to predict which customers are at high risk of churning.

We will use Google Colaboratory (Colab), a free online tool, and highlight its built-in code generation features, which can help speed up the process, especially for those less familiar with programming. However, the focus will be on understanding the *concepts* and the *business implications*.

Dataset: IBM Telco Customer Churn dataset. (<https://raw.githubusercontent.com/IBM/telco-customer-churn-on-icp4d/refs/heads/master/data/Telco-Customer-Churn.csv>)

2 Setting Up: Loading Libraries and Data

Before we can analyze anything, we need to load the necessary tools and the data itself. In data science, the `pandas` library is the standard tool for working with tabular data (like spreadsheets).

2.1 Loading the Data

We instruct Colab to load the dataset directly from the web URL into a structure called a *DataFrame* (think of it as a smart spreadsheet).

```
1 # prompt: load the dataset from this link to pandas dataframe: [URL]
2
3 import pandas as pd # Import the pandas library
4
5 # Load the dataset into a pandas DataFrame
6 try:
7     url = 'https://raw.githubusercontent.com/IBM/telco-customer-churn-on-icp4d/refs/heads/master/data/Telco-Customer-Churn.csv'
8     df = pd.read_csv(url)
9     print("Dataset loaded successfully.")
10 except Exception as e:
11     print(f"Error loading dataset: {e}")
12     df = None # Set df to None if loading fails
13
14 # Display the first 5 rows
15 if df is not None:
```

```
print(df.head())
```

Listing 1: Loading data using pandas

The `df.head()` command shows the first few rows, giving us an initial glimpse of the columns and data types. We see columns like `gender`, `SeniorCitizen`, `tenure` (how long they've been a customer), `MonthlyCharges`, and crucially, `Churn`.

3 Getting to Know Your Data (Exploratory Data Analysis - Part 1)

Understanding your data is the most crucial step. We need to know what information we have, its quality, and its basic characteristics. Colab's code generation can quickly provide answers to common questions.

3.1 Basic Checks

We ask Colab to examine the DataFrame (`df`) to understand:

1. What columns (fields) are available?
2. Are there any missing values? Missing data can cause problems for analysis and models.
3. What are the typical values for numeric fields (like average tenure or monthly charges)?
4. What categories exist in non-numeric fields (like different contract types or payment methods)?

```
1 # prompt: examine the dataset df to (1) see what fields/columns are available
2 # (2) check if there are missing data (3) check the value range for numeric data
3 # and categories for categorical/discrete data (4) other descriptive statistics
```

Listing 2: Prompt for initial data exploration

The generated code (not fully shown here for brevity, but present in the notebook) would typically use commands like:

- `df.columns`: Lists all column names.
- `df.isnull().sum()`: Counts missing values in each column.
- `df.describe()`: Shows statistics (count, mean, min, max, etc.) for numeric columns.
- `df['column_name'].value_counts()`: Shows the frequency of each category in a categorical column.

3.2 Interpreting the Findings

Running these checks reveals important details (summarized from the notebook's analysis):

- **Columns:** We have about 21 columns covering customer demographics, account information (tenure, contract, payment method), services subscribed to (phone, internet, streaming, etc.), charges, and the churn status.

- **Missing Data:** Initially, it appears there's no missing data reported by `isnull().sum()`. However, closer inspection (often needed!) reveals that the `TotalCharges` column contains empty strings for some customers, which represent missing information and need handling later.
- **Numeric Ranges:** We see the range of `tenure` (0-72 months) and `MonthlyCharges`.
- **Categories:** We see the different types within columns like `Contract` ('Month-to-month', 'One year', 'Two year'), `PaymentMethod`, etc. Some columns have values like 'No phone service' or 'No internet service', which might require special attention.
- **Target Variable:** The `Churn` column ('Yes'/'No') is our target – what we want to predict. We see there's an imbalance: more customers haven't churned ('No') than have ('Yes'). This is common in churn datasets.

A summary table helps organize these findings (adapted from the notebook):

Table 1: Initial Data Understanding Summary

Column Name	Data Type	Example Values / Range	Analysis Notes
customerID	Text	7590-VHVEG	Unique ID, likely not useful for prediction.
gender	Text	Female, Male	Categorical, fairly balanced.
SeniorCitizen	Number (0/1)	0, 1	Binary indicator (Yes/No).
Partner	Text	Yes, No	Categorical.
Dependents	Text	Yes, No	Categorical, more 'No'.
tenure	Number	0 - 72 (Avg: 32.4)	Customer relationship duration (months).
Contract	Text	Month-to-month, ...	Categorical. Seems important.
MonthlyCharges	Number	18.25 - 118.75 (Avg: 64.8)	Continuous numeric value.
TotalCharges	Text/Number	'29.85', '', '1889.5', ...	Issue: Stored as text, has empty strings.
Churn	Text	Yes, No	Target variable , imbalanced.
...	...	...	(Other service columns)

4 Visualizing the Data (Exploratory Data Analysis - Part 2)

Numbers and tables are useful, but visualizations often reveal patterns more clearly. We can ask Colab to generate common plots.

4.1 Histogram: Distribution of Tenure

A histogram shows how frequently different values occur for a single numeric variable.

```
1 # prompt: plot histogram of tenure column from df
```

Listing 3: Prompt for plotting a histogram

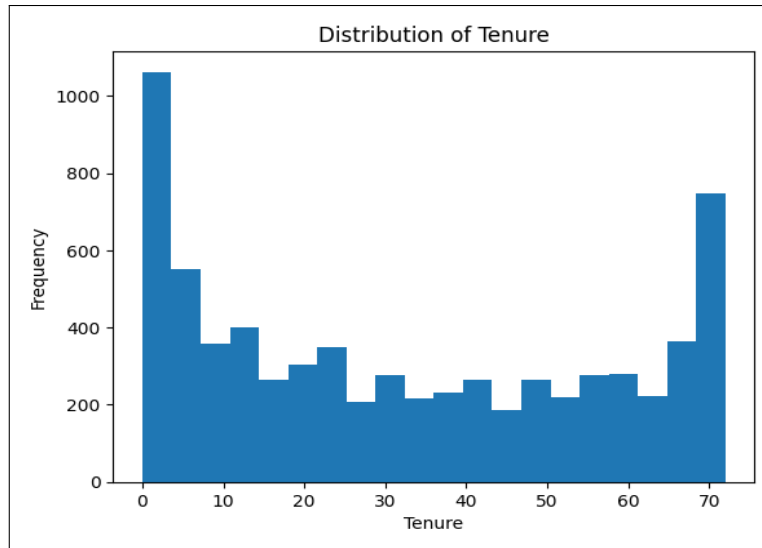


Figure 1: Distribution of Customer Tenure (Months). Shows many new customers and many long-term customers.

The histogram for `tenure` (Figure 1) often shows a "U-shape" distribution in telco data: many customers are either very new (low tenure) or have been with the company for a long time (high tenure).

4.2 Box Plot: Monthly Charges vs. Churn

A box plot is great for comparing the distribution of a numeric variable across different categories. Here, we compare monthly charges for customers who churned versus those who didn't.

```
1 # prompt: plot boxplot of MonthlyCharges grouped by Churn in df
```

Listing 4: Prompt for plotting a box plot

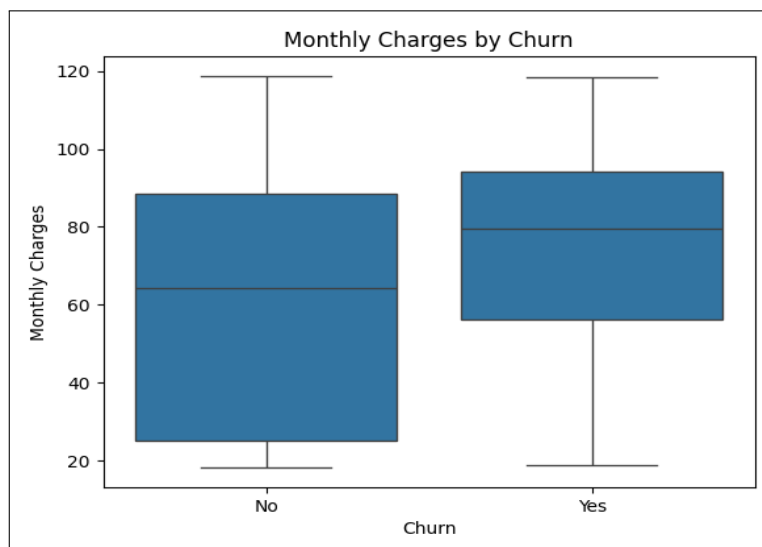


Figure 2: Box Plot of Monthly Charges by Churn Status. Suggests customers who churn tend to have higher monthly charges.

The box plot (Figure 2) typically shows that customers who churned ('Yes') tend to have higher median monthly charges compared to those who stayed ('No'). The "box" shows the middle 50

Other useful plots (mentioned in notebook prompts but plots not shown) include bar charts for categorical variables (like Churn distribution) or count plots (like Contract type colored by Churn status).

5 Preparing Data for Modeling (Preprocessing)

Real-world data is rarely perfect. Before building predictive models, we need to clean and transform it.

5.1 Handling Data Issues

Based on our initial exploration:

1. **Convert TotalCharges to Numeric:** This column was identified as text and contained empty strings. We need to convert it to numbers. Empty strings likely represent customers with zero tenure (brand new), so we might fill these missing numeric values with 0. Errors during conversion should be handled (e.g., set to 'Not a Number' or NaN, then filled).
2. **Drop Irrelevant Columns:** The `customerID` is unique to each customer and doesn't provide predictive power for **future** churn based on **patterns**. We remove it.

5.2 Encoding Categorical Data: One-Hot Encoding

Most machine learning models work with numbers, not text categories (like 'Female'/'Male' or 'Month-to-month'/'Two year'). We need to convert these text categories into a numerical format. A common technique is **One-Hot Encoding**.

Imagine the `Contract` column:

- Month-to-month
- One year
- Two year

One-Hot Encoding transforms this single column into *multiple* new binary (0/1) columns:

- `Contract_One year` (1 if true, 0 otherwise)
- `Contract_Two year` (1 if true, 0 otherwise)

(Note: We often drop one category, like 'Month-to-month' here, to avoid redundancy; if both new columns are 0, it implies the contract was month-to-month. This is called `drop_first=True`).

We apply this to all relevant categorical columns, including our target variable `Churn` (transforming 'Yes'/'No' to 1/0).

```
1 # prompt: help me create the clean dataset df_clean based on the below information
  :
2 # <paste data summary table from section 3.2>
3 # --> Convert TotalCharges to numeric, fill missing with 0
4 # --> Drop customerID
5 # --> One-hot encode categorical columns [list columns] including Churn
```

Listing 5: Prompt for cleaning and encoding (Conceptual)

The generated code uses ‘pd.to_numeric’, ‘fillna(0)’, ‘df.drop()’, and ‘pd.get_dummies’ to perform these steps, resulting in a cleaned DataFrame ‘df_clean’ ready for modeling.

6 Understanding Relationships (Correlation)

Correlation measures the linear relationship between two numeric variables. It ranges from -1 (perfect negative relationship) to +1 (perfect positive relationship), with 0 indicating no linear relationship. A **heatmap** is a graphical way to visualize the correlation matrix for many variables at once.

```
1 # prompt: plot correlation heatmap for numerical columns in df_clean
```

Listing 6: Prompt for correlation heatmap

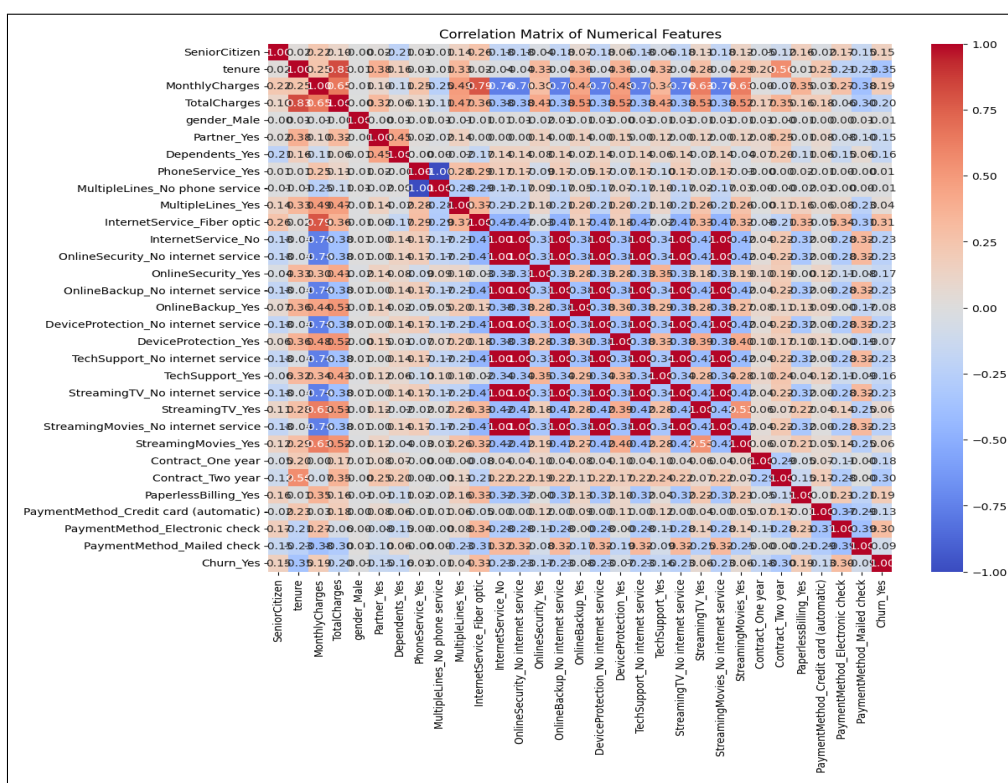


Figure 3: Correlation Heatmap of Features (after encoding). Red indicates positive correlation, blue indicates negative.

The heatmap (Figure 3) helps identify:

- **Strong Relationships:** Bright red or blue squares indicate strong correlations. For example, we often see a strong positive correlation between **tenure** and **TotalCharges**.
- **Relationships with Churn:** Look at the row/column for **Churn_Yes**. We might see that ‘tenure’ has a negative correlation (longer tenure, less likely to churn), while ‘Contract_Month-to-month’ (represented implicitly or by other contract types being 0) might have a positive correlation (more likely to churn). High ‘MonthlyCharges’ often correlates positively with churn.

- **Multicollinearity:** High correlation between predictor variables (e.g., maybe between different internet service options) can sometimes be an issue for certain models, though often acceptable.

7 Simplifying Data (Dimensionality Reduction with PCA)

Our dataset now has many columns, especially after one-hot encoding. Sometimes, having too many columns ("high dimensionality") can make modeling slow or less effective (the "curse of dimensionality"). **Principal Component Analysis (PCA)** is a technique to reduce the number of columns (dimensions) while trying to retain most of the important information (variance) in the data.

Think of it as creating new, synthetic columns (Principal Components) that are combinations of the original ones, ordered by how much information they capture. We can then keep just the first few principal components.

```
1 # prompt: perform PCA with 2 components on df_clean
```

Listing 7: Prompt for performing PCA

PCA is often used for:

- **Visualization:** Reducing data to 2 or 3 dimensions allows us to plot it and potentially see clusters or separation between groups (like churned vs. non-churned customers).
- **Faster Modeling:** Using fewer PCA components instead of all original features can speed up model training.

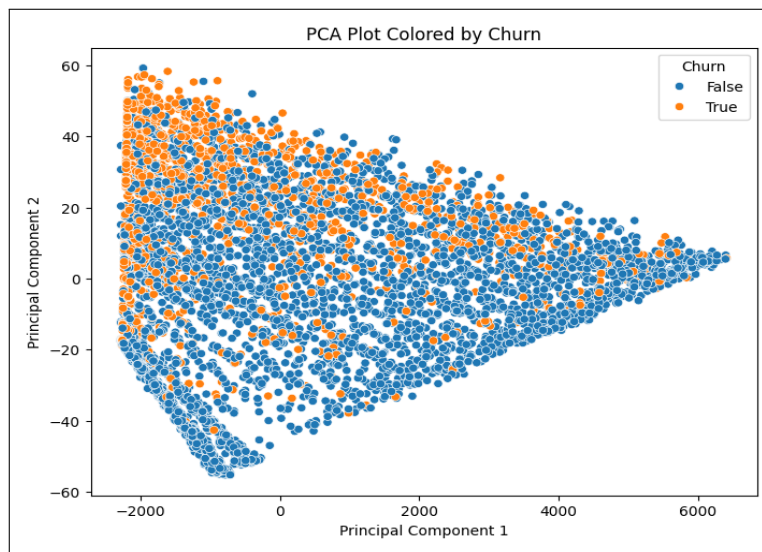


Figure 4: PCA Plot (First two components) colored by Churn Status. We check if the two groups separate visually.

Figure 4 shows the data points plotted using their first two principal components. If the blue ('No Churn') and red ('Yes Churn') dots are well-separated, it suggests the data might be easier to model. Often, the separation isn't perfect, indicating the complexity of the churn problem.

The notebook also shows PCA "loadings" (Figure in notebook page 8), which tell us how much each original feature contributes to each principal component. This can help interpret what the components represent (e.g., PC1 might be related to 'TotalCharges', PC2 to 'MonthlyCharges').

Note: For this tutorial, we proceed to model using the full 'df_clean' data, not the PCA-reduced data, but PCA is a valuable tool to know.

8 Building Predictive Models

Now we build models to predict the `Churn_Yes` variable based on the other features.

8.1 Train-Test Split: The Golden Rule

Before training *any* model, we must split our data into two sets:

- **Training Set (e.g., 80% of data):** Used to teach the model the patterns associated with churn.
- **Testing Set (e.g., 20% of data):** Held back and used *only* at the very end to evaluate how well the trained model performs on *unseen* data. This gives a realistic estimate of its performance in the real world.

If we evaluate the model on the same data it was trained on, it will likely look much better than it actually is (overfitting). We use 'stratify=y' to ensure both sets have a similar proportion of churned/non-churned customers as the original dataset.

```
1 # prompt: split X and y into training and testing sets with test size 0.2
2 # and random state 42
3 # (Assuming X = df_clean without Churn, y = df_clean['Churn_Yes'])
```

Listing 8: Prompt for Train-Test Split

8.2 Model 1: Logistic Regression

Logistic Regression is a fundamental statistical model often used for binary classification (Yes/No outcomes). It estimates the probability that a customer will churn based on their features.

```
1 # prompt: train logistic regression model on X_train and y_train
2 # prompt: evaluate logistic regression model using accuracy score on y_test
3 # and predictions
4 # prompt: plot confusion matrix for logistic regression
```

Listing 9: Prompt for training Logistic Regression

Evaluation Metrics: How do we know if the model is good?

- **Accuracy:** Overall percentage of correct predictions. Simple, but can be misleading if the data is imbalanced (e.g., predicting 'No Churn' for everyone might give high accuracy if only 10
- **Confusion Matrix:** A table showing the details of correct and incorrect predictions.
 - **True Positives (TP):** Correctly predicted 'Yes Churn'.
 - **True Negatives (TN):** Correctly predicted 'No Churn'.

- **False Positives (FP):** Incorrectly predicted 'Yes Churn' (predicted churn, but didn't). (Type I Error)
- **False Negatives (FN):** Incorrectly predicted 'No Churn' (predicted no churn, but did). (Type II Error) - *Often the most costly error in churn prediction!*
- **Classification Report:** Provides more nuanced metrics:
 - **Precision:** Of those predicted to churn, how many actually did? $(TP / (TP + FP))$. High precision means fewer false alarms.
 - **Recall (Sensitivity):** Of those who actually churned, how many did we catch? $(TP / (TP + FN))$. High recall means we miss fewer churning customers.
 - **F1-Score:** Harmonic mean of Precision and Recall, providing a single score balancing both.

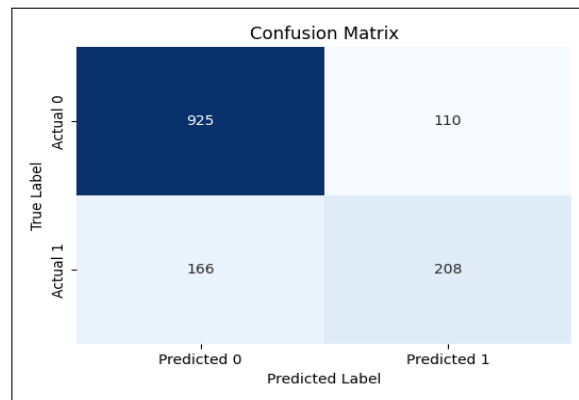


Figure 5: Example Confusion Matrix for Logistic Regression

The notebook shows Logistic Regression achieved around 80.4% accuracy, with decent precision and recall (though details vary). The confusion matrix (similar to Fig 5) visualizes the TP, TN, FP, FN counts.

8.3 Model 2: Random Forest

Random Forest is a more complex model that builds many Decision Trees (explained next) and combines their predictions. It's often more accurate and robust than a single tree or logistic regression.

```
1 # prompt: train random forest classifier on X_train and y_train
2 # (Evaluation prompts are similar to Logistic Regression)
```

Listing 10: Prompt for training Random Forest

The notebook shows Random Forest achieved around 78.5

8.4 Model 3: Decision Tree

A Decision Tree creates a flowchart-like structure of questions based on features to arrive at a prediction. They are highly interpretable. We can limit the depth ('max_depth') to prevent overfitting and keep the tree simpler.

```

1 # prompt: train decision tree classifier on X_train and y_train with max depth of
  4
2 # (Evaluation prompts are similar)

```

Listing 11: Prompt for training Decision Tree

The notebook shows the Decision Tree (with ‘max_depth=4’) achieved around 79.6% accuracy, performing reasonably well and offering interpretability.

8.5 Interpreting the Decision Tree: Finding Business Rules

The great advantage of Decision Trees is that we can extract the exact rules they use.

```

1 # prompt: create a function to extract a list of if-then rules from the
2 # decision tree above.

```

Listing 12: Prompt for extracting Decision Tree rules

The generated rules look like: IF tenure <= 16.5 AND InternetService_Fiber optic > 0.5 AND TotalCharges <= 120.0 ... THEN [[Churn Probabilities]]

These rules directly translate into business insights: "Customers with low tenure, using fiber optic internet, and with low total charges so far seem likely/unlikely to churn...". These rules can potentially be translated into Excel formulas or used by marketing teams to identify at-risk segments.

9 Using Colab’s Code Generation

Throughout this tutorial, we’ve mentioned prompts used with Colab’s code generation feature (often accessed via a "Generate" button or sparkle icon in code cells). This AI assistant helps by:

- Writing boilerplate code for common tasks (loading data, plotting, modeling).
- Suggesting relevant libraries and functions.
- Speeding up the workflow, especially for repetitive tasks.

Important: Always review the generated code. It’s a helpful assistant, but not infallible. Ensure it does what you intend and is appropriate for your specific task. You might need to refine the prompt or edit the code.

10 Conclusion: From Data to Business Action

We walked through a data science workflow to predict customer churn:

1. Explored the data to understand customer characteristics and identify potential issues.
2. Prepared the data by handling missing values and converting formats.
3. Visualized relationships using plots and heatmaps.
4. Built and evaluated several predictive models (Logistic Regression, Random Forest, Decision Tree).
5. Interpreted model results, focusing on metrics relevant to the business (like recall for catching churners) and extracting interpretable rules from the Decision Tree.

Key factors often linked to churn in this dataset include:

- **Contract Type:** Month-to-month contracts churn much more often.
- **Tenure:** Newer customers are more likely to churn.
- **Internet Service:** Fiber optic customers sometimes show higher churn (perhaps related to price or service issues).
- **Monthly Charges:** Higher charges often correlate with higher churn.

Colab's code generation facilitated many steps, but the core task remains understanding the data, the methods, and the business context. While AI can write code, domain knowledge and critical thinking are essential to interpret results and translate them into actionable business strategies for customer retention.